

LAB 9: GIAO TIẾP SDRAM & SRAM

1. MỤC ĐÍCH

Thông qua bài thực hành này, sinh viên sẽ hiểu rõ:

- Cấu trúc của SRAM, SDRAM và PLL.
- Cách xây dựng hệ thống phần cứng SRAM, SDRAM và PLL trên Qsys.
- Cách xây dựng chương trình C trên công cụ Nios II – Eclipse Platform để điều khiển các khối phần cứng đã thiết kế trước đó.

2. TỔNG QUAN

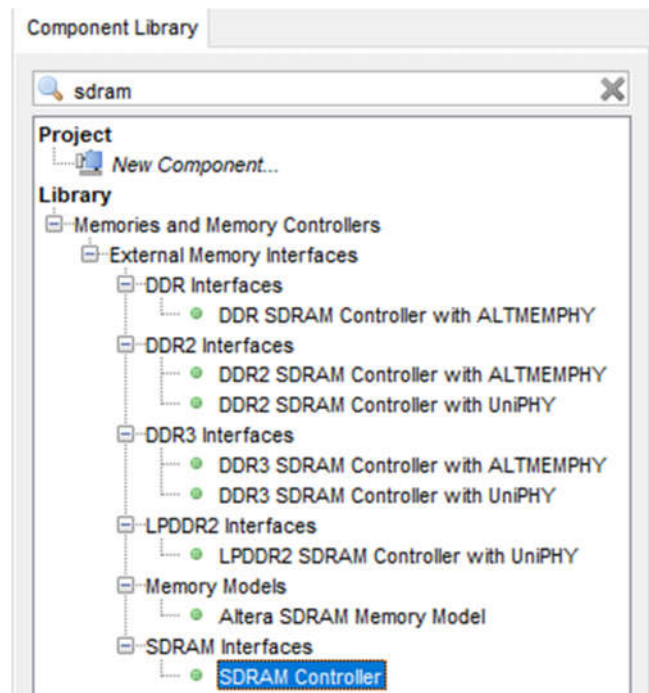
Hướng dẫn thực hiện

Thực hiện tạo Qsys với các module sau:

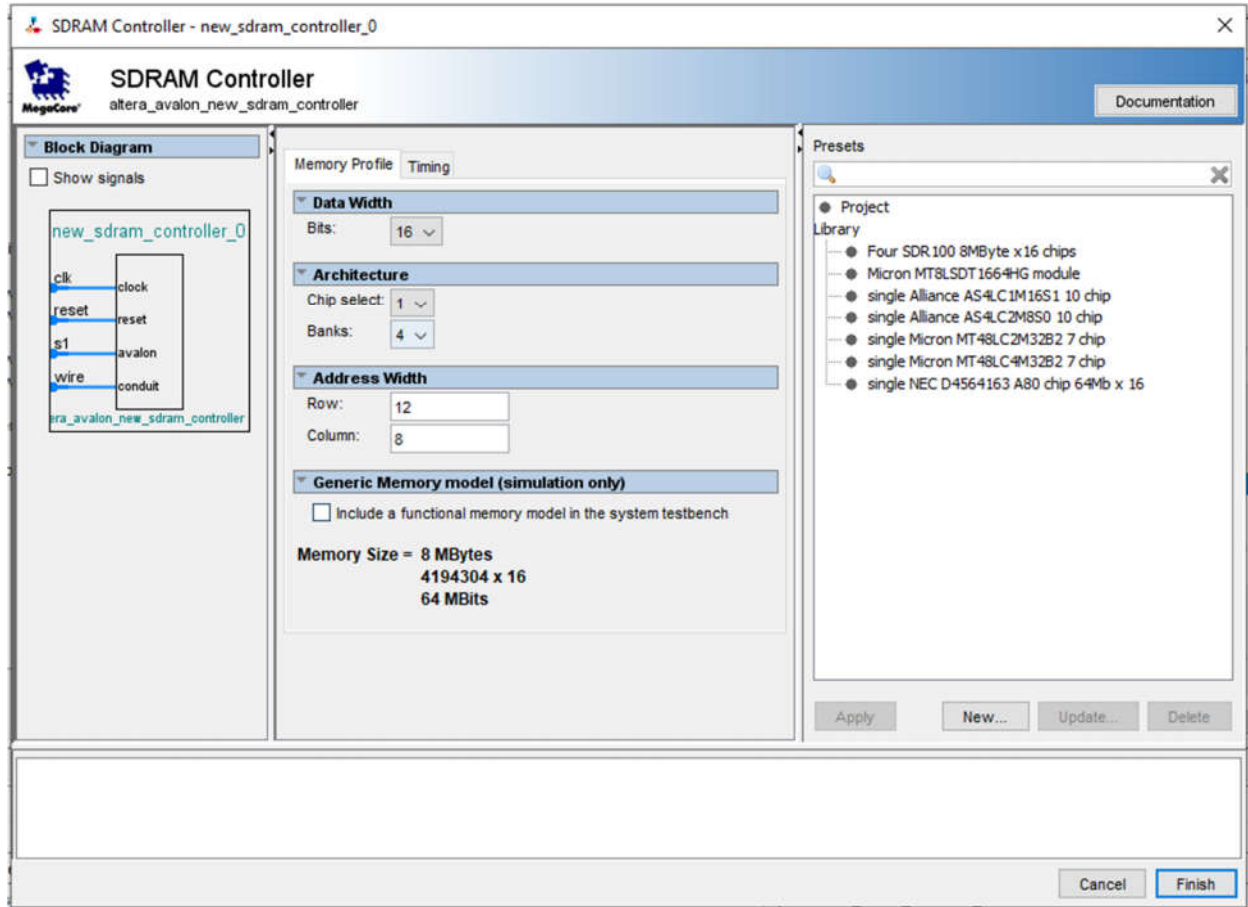
- *Clock 50M*
- *Nios II/e*
- *Jtag UART*
- *SDRAM*

Lưu ý: trong bài tập này, chúng ta sẽ sử dụng bộ nhớ SDRAM để thay thế cho On-Chip Memory, vì vậy mà ta không sử dụng On-chip Memory.

❖ **Add lõi IP: SDRAM**



Cấu hình cài đặt cho SDRAM có Data là 16 bits



❖ Thực hiện kết nối các IP như hình sau

System Contents									
Address Map									
Clock Settings									
Project Settings									
Instance Parameters									
System Inspector									
HDL Example									
Generation									
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Opoc
☒		clk_0	Clock Source	clk	clk_0				
☒		clk_in	Clock Input	Double-click to export					
☒		clk_in_reset	Reset Input	Double-click to export					
☒		clk	Clock Output	Double-click to export					
☒		clk_reset	Reset Output	Double-click to export					
☒		nios2_qsys_0	Nios II Processor						
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset_n	Reset Input	Double-click to export	[clk]				
☒		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
☒		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
☒		jtag_debug_module_re	Reset Output	Double-click to export	[clk]				
☒		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]				
☒		custom_instruction_m	Custom Instruction Master	Double-click to export	[clk]				
☒		jtag_uart_0	JTAG UART						
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	[clk]				
☒		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0200_0800	0x0200_0fff		
☒		sdr	SDRAM Controller						
☒		clk	Clock Input	Double-click to export	clk_0				
☒		reset	Reset Input	Double-click to export	[clk]				
☒		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0200_1008	0x0200_100f		
☒		wire	Conduit	sdr_wire					

❖ Thực hiện chọn Reset Vector và Exception Vector cho NIOS là: SDRAM



Nios II Processor

altera_nios2_qsys

Block Diagram

☐ Show signals

Nios II

Selector Guide

	RISC 32-bit	RISC 32-bit Instruction Cache Branch Prediction Hardware Multiply Hardware Divide	RISC 32-bit Instruction Cache Branch Prediction Hardware Multiply Hardware Divide Barrel Shifter Data Cache Dynamic Branch Prediction
Memory Usage (e.g. Stratix IV)	Two M9Ks (or equiv.)	Two M9Ks + cache	Three M9Ks + cache

Hardware Arithmetic Operation

Hardware multiplication type: Embedded Multipliers

☐ Hardware divide

Reset Vector

Reset vector memory: sdram.s1

Reset vector offset: 0x00000000

Reset vector: 0x00000000

Exception Vector

Exception vector memory: sdram.s1

Exception vector offset: 0x00000020

Exception vector: 0x00000020

Thực hiện Generate cho Qsys.

❖ Thực hiện code verilog file Top-Level như sau

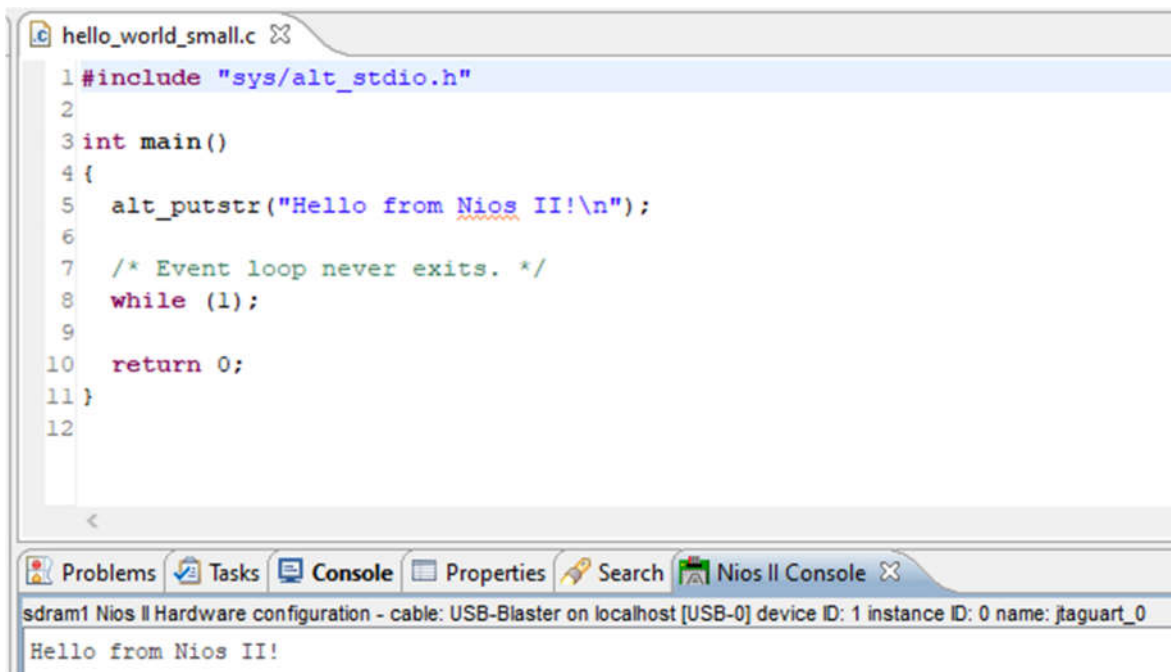
```

1  module sdram (CLOCK_50, DRAM_CLK, DRAM_CKE,
2      DRAM_ADDR, DRAM_BA_1, DRAM_BA_0, DRAM_CS_N, DRAM_CAS_N, DRAM_RAS_N,
3      DRAM_WE_N, DRAM_DQ, DRAM_UDQM, DRAM_LDQM);
4      input CLOCK_50;
5      output [11:0] DRAM_ADDR;
6      output DRAM_BA_1, DRAM_BA_0, DRAM_CAS_N, DRAM_RAS_N, DRAM_CLK;
7      output DRAM_CKE, DRAM_CS_N, DRAM_WE_N, DRAM_UDQM, DRAM_LDQM;
8      inout [15:0] DRAM_DQ;
9
10     nios_sys u0 (
11         .clk_clk (CLOCK_50),           // clk.clk
12         .sdram_wire_addr (DRAM_ADDR),  // sdram_wire.addr
13         .sdram_wire_ba ({DRAM_BA_1, DRAM_BA_0}), // .ba
14         .sdram_wire_cas_n (DRAM_CAS_N), // .cas_n
15         .sdram_wire_cke (DRAM_CKE),     // .cke
16         .sdram_wire_cs_n (DRAM_CS_N),   // .cs_n
17         .sdram_wire_dq (DRAM_DQ),       // .dq
18         .sdram_wire_dqm ({DRAM_UDQM, DRAM_LDQM}), // .dqm
19         .sdram_wire_ras_n (DRAM_RAS_N), // .ras_n
20         .sdram_wire_we_n (DRAM_WE_N)    // .we_n
21     );
22
23     assign DRAM_CLK = CLOCK_50;
24 endmodule

```

Biên dịch và nạp file SOF lên KIT.

❖ Thực hiện và biên dịch code C trên Eclipse



The screenshot displays the Nios II IDE interface. The top pane shows the source code for `hello_world_small.c`. The code includes `sys/alt_stdio.h`, defines a `main` function that prints "Hello from Nios II!\n", enters an infinite loop, and returns 0. The bottom pane shows the "Nios II Console" output, which includes the hardware configuration details and the printed message "Hello from Nios II!".

```
1 #include "sys/alt_stdio.h"
2
3 int main()
4 {
5     alt_putstr("Hello from Nios II!\n");
6
7     /* Event loop never exits. */
8     while (1);
9
10    return 0;
11 }
12
```

Problems Tasks Console Properties Search Nios II Console

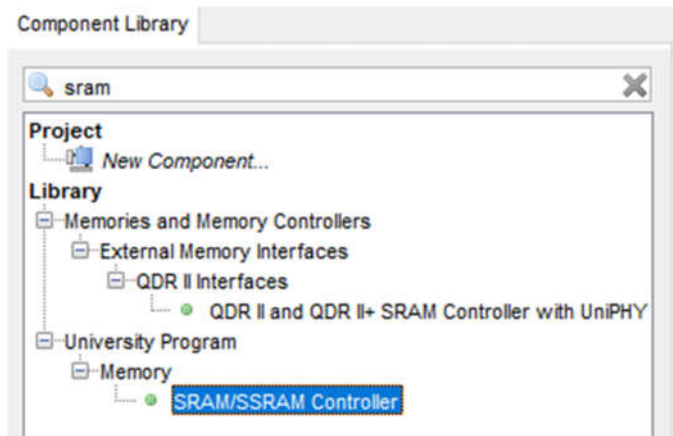
sdr1 Nios II Hardware configuration - cable: USB-Blaster on localhost [USB-0] device ID: 1 instance ID: 0 name: jtaguart_0

Hello from Nios II!

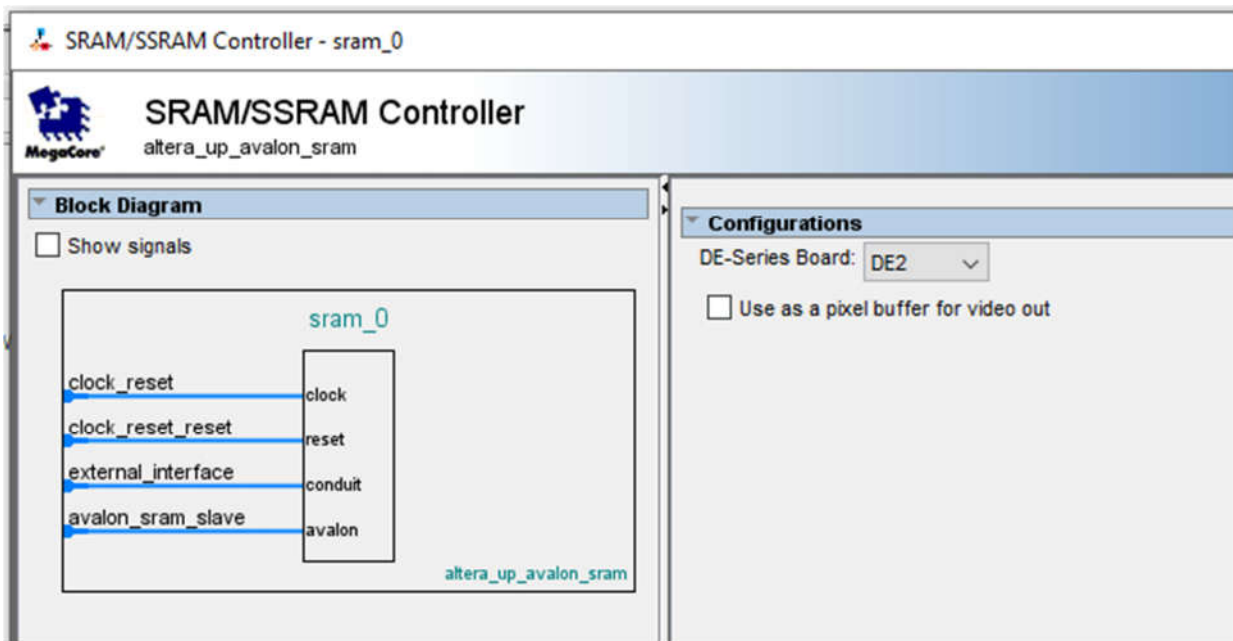
Bài tập ví dụ 2: SRAM - SDRAM – PLL

Hướng dẫn thực hiện

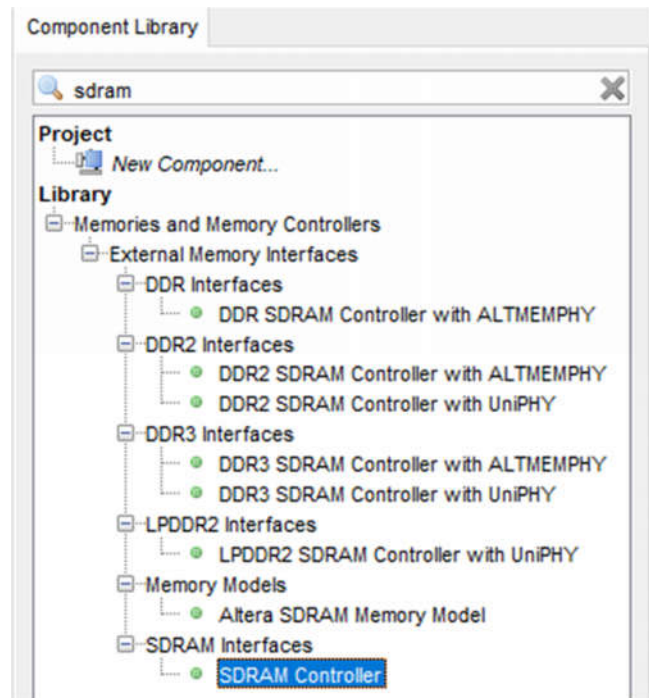
- ❖ Add các lõi IP hệ thống Nios:
 - Nios II/e
 - Clock
 - On-Chip Memory: 16 KB
 - Jtag-UART
 - Sys ID
- ❖ Thêm lõi IP: **SRAM**



Cấu hình cho SRAM: Chọn Board DE2 như hình sau:

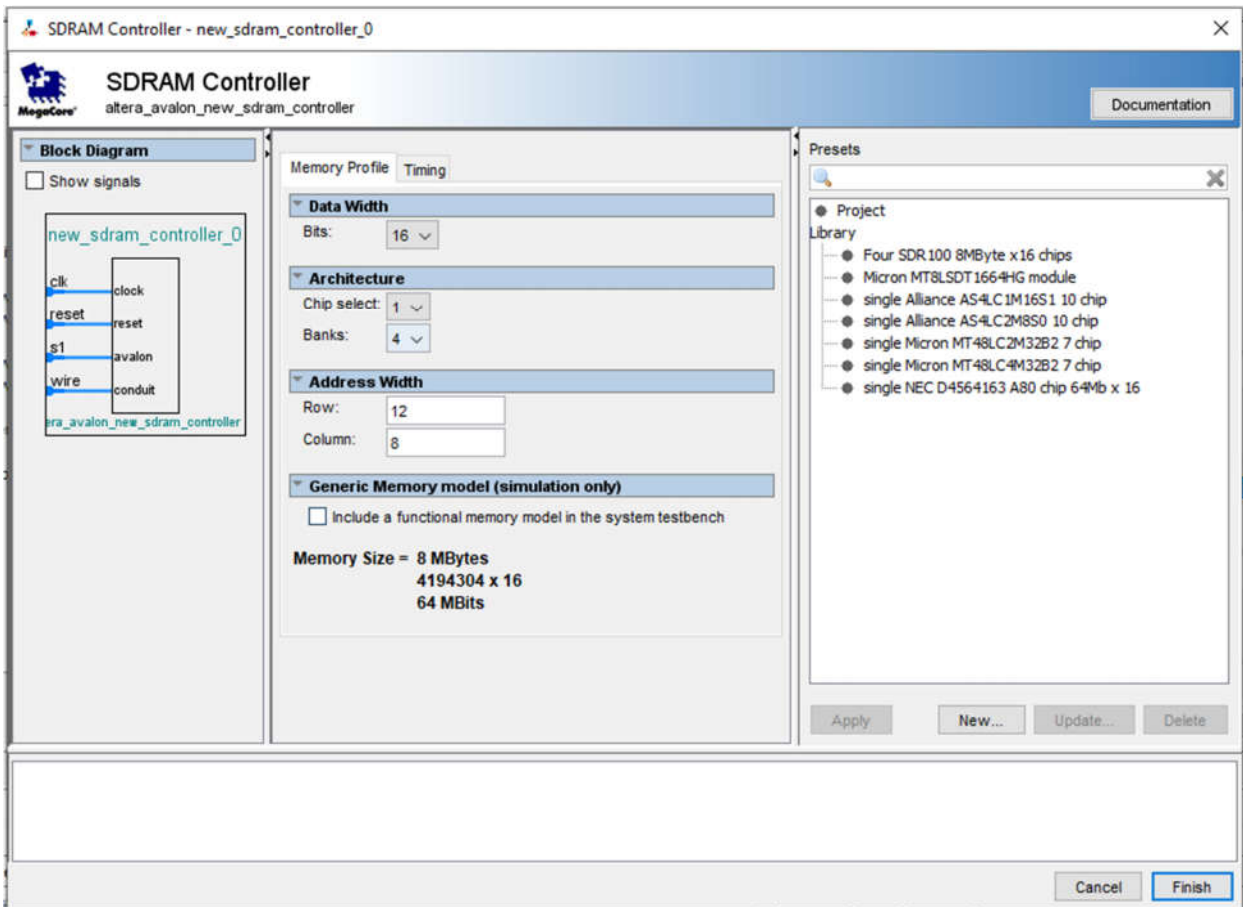


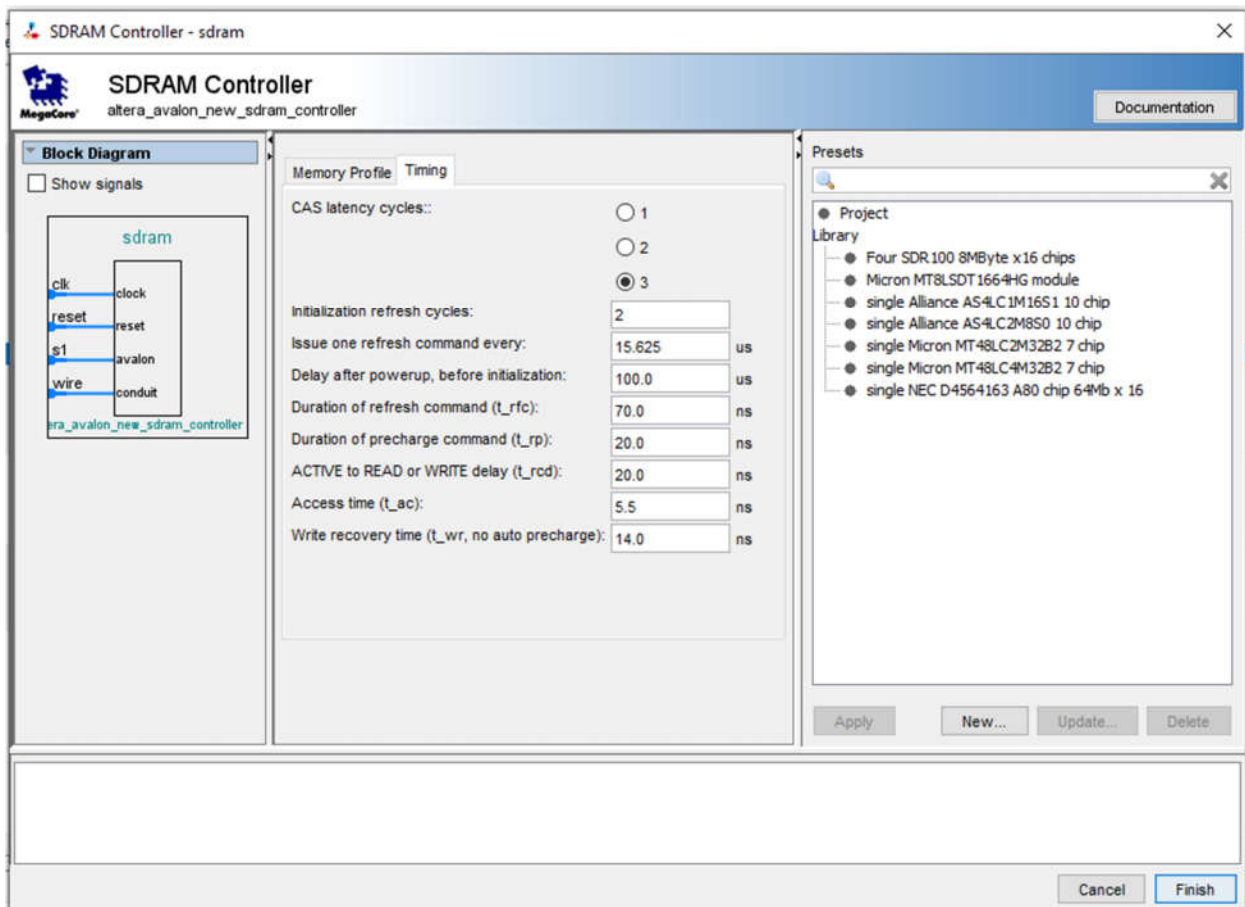
- ❖ Add thêm lõi IP: **SDRAM**



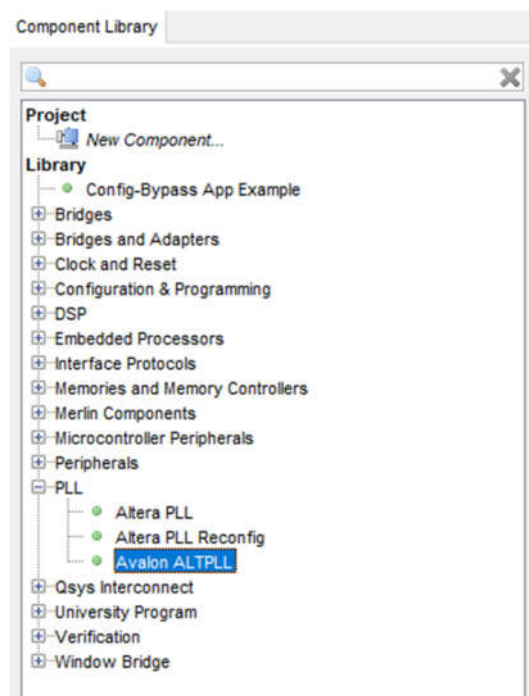
Cấu hình Cài đặt SDRAM có Data là **16 bits**.

Như vậy, Tổng Memory của SDRAM = 8 Mbytes x 16 = 64 Mbytes.





- ❖ Add Thêm lõi IP vòng khóa pha (Phase Lock Loop) ALTPLL như sau:



Cấu hình cho ALTPLL: cài đặt tần số xung **clock input 50 Mhz** như sau:

MegaWizard Plug-In Manager [page 1 of 7]

ALTPLL

1 Parameter Settings 2 Output Clocks 3 EDA

General/Modes Inputs/Lock Clock switchover

Currently selected device family: Cyclone II

☒ Match project/default

ALTPLL1666840690864052

inclk0 inclk0 frequency: 50.000 MHz
Operation Mode: Normal

Clk	Ratio	Ph (deg)	DC (%)
c0	1/1	0.00	50.00
c1	1/1	-54.00	50.00

Cyclone II

Able to implement the requested PLL

General

Which device speed grade will you be using? 6

☐ Use military temperature range devices only

What is the frequency of the inclk0 input? 50.000 MHz

☐ Set up PLL in LVDS mode Data rate: Not Available Mbps

PLL Type

Which PLL type will you be using?

☐ Fast PLL ☐ Enhanced PLL ☒ Select the PLL type automatically

Operation Mode

How will the PLL outputs be generated?

☒ Use the feedback path inside the PLL

☒ In normal mode

☐ In source-synchronous compensation Mode

☐ In zero delay buffer mode

☐ Connect the fbmimic port (bidirectional)

☐ With no compensation

☐ Create an 'fbmimic' input for an external feedback (External Feedback Mode)

Which output clock will be compensated for? c0

Cancel < Back Next > Finish

Chuyển sang Tab: Output Clock. Thực hiện cài đặt nguồn xung Clock CLK_C0:

Cài đặt nguồn xung **Clock output C0** để sử dụng cho tất cả hệ thống các lõi IP trong Qsys như sau:

ALTPLL

1 Parameter Settings 2 **Output Clocks** 3 EDA

clk c0 > clk c1 > clk c2 >

ALTPLL1666840690864052

inc0 frequency: 50.000 MHz
Operation Mode: Normal

Clk	Ratio	Ph (deg)	DC (%)
c0	1/1	0.00	50.00
c1	1/1	-54.00	50.00

Cyclone II

c0 - Core/External Output Clock
Able to implement the requested PLL

☒ Use this clock

Clock Tap Settings

☐ Enter output clock frequency:

☒ Enter output clock parameters:

Clock multiplication factor

Clock division factor

Clock phase shift

Clock duty cycle (%)

Note: The displayed internal settings of the PLL is recommended for use by advanced users only

Requested Settings	Actual Settings
100.00000000 MHz	50.000000
1	1
1	1
0.00 deg	0.00
50.00	50.00

Description

Description	Val
Primary clock VCO frequency (MHz)	75..
Modulus for M counter	15

Per Clock Feasibility Indicators

c0 c1 c2

Cancel < Back Next > Finish

Thực hiện cài đặt nguồn xung Clock **CLK_C1**: Cài đặt nguồn xung **Clock output C1** nguồn xung clock này sẽ sử dụng để cấp vào cho SDRAM, với **Clock phase Shift= -3ns**, như sau:

ALTPLL

1 Parameter Settings 2 Output Clocks 3 EDA

clk c0 > **clk c1** > clk c2

ALTPLL1666840690864052

inclk0 inclk0 frequency: 50.000 MHz
Operation Mode: Normal

Clk	Ratio	Ph (deg)	DC (%)
c0	1/1	0.00	50.00
c1	1/1	-54.00	50.00

Cyclone II

c1 - Core/External Output Clock
Able to implement the requested PLL

☒ Use this clock

Clock Tap Settings

☐ Enter output clock frequency:
☐ **Enter output clock parameters:**

Clock multiplication factor: 1

Clock division factor: 1

Clock phase shift: -3.00 ns

Clock duty cycle (%): 50.00

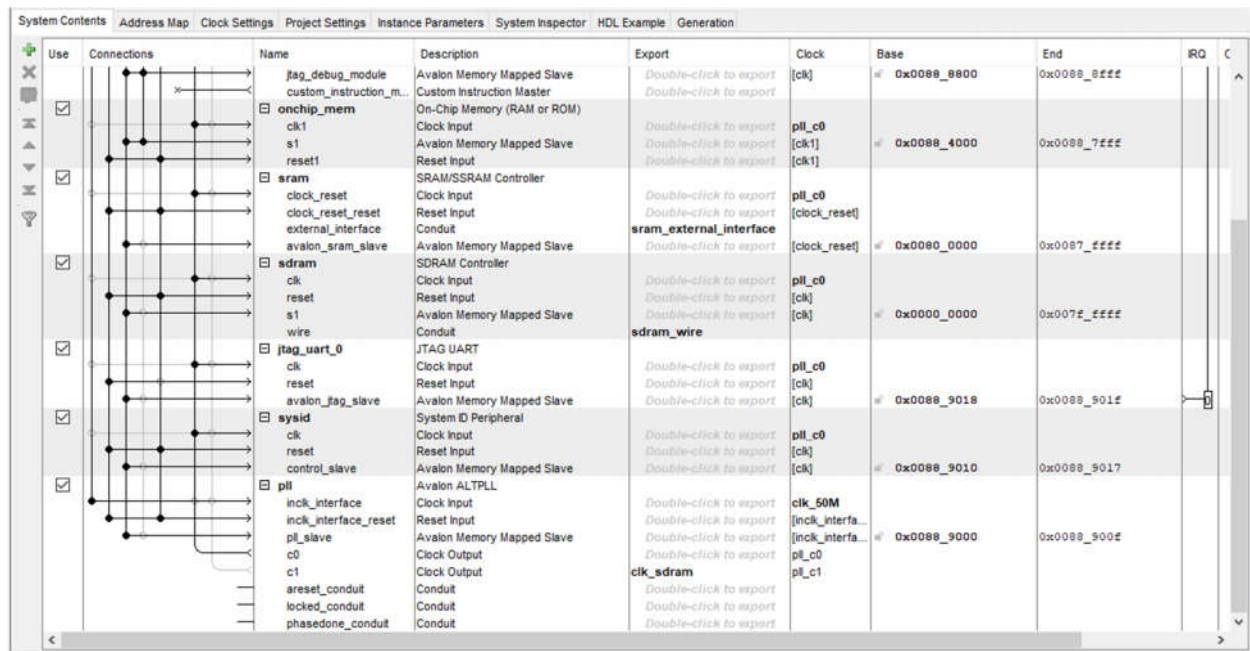
Note: The displayed internal settings of the PLL is recommended for use by advanced users only

Description	Val
Primary clock VCO frequency (MHz)	75.000000
Modulus for M counter	15

Per Clock Feasibility Indicators
c0 c1 c2

Cancel < Back Next > Finish

- ❖ Thực hiện kết nối các IP đúng như hình sau, để nguồn xung CLOCK 50M sẽ cấp vào cho ALTPLL. Sau đó, ALTPLL sẽ tạo ra 2 nguồn Clock, nguồn Clock C0 sẽ cấp cho toàn hệ thống các lõi IP. Và nguồn Clock C1 sẽ cấp vào cho SDRAM.



Sau đó Generate Qsys.

❖ Thực hiện Code verilog file Top-Level như sau để gán các Input/Output cho SRAM và SDRAM.

```

1 module sram_sdram ( input CLOCK_50,
2   // to/from SRAM
3   output [17:0]SRAM_ADDR,
4   output SRAM_CE_N, SRAM_OE_N, SRAM_WE_N,
5   output SRAM_LB_N, SRAM_UB_N,
6   inout [15:0]SRAM_DQ,
7   //to/from SDRAM side
8   output [11:0] DRAM_ADDR,
9   output DRAM_BA_1, DRAM_BA_0, DRAM_CAS_N, DRAM_RAS_N, DRAM_CLK,
10  output DRAM_CKE, DRAM_CS_N, DRAM_WE_N, DRAM_UDQM, DRAM_LDQM,
11  inout [15:0] DRAM_DQ
12 );
13
14 nios_sys u0 (
15   .clk_clk          (CLOCK_50),          // clk.clk
16   .clk_sdram_clk    (DRAM_CLK),          // clk_sdram.clk
17   .sram_external_interface_DQ (SRAM_DQ), // sram_external_interface.DQ
18   .sram_external_interface_ADDR (SRAM_ADDR), // .ADDR
19   .sram_external_interface_LB_N (SRAM_LB_N), // .LB_N
20   .sram_external_interface_UB_N (SRAM_UB_N), // .UB_N
21   .sram_external_interface_CE_N (SRAM_CE_N), // .CE_N
22   .sram_external_interface_OE_N (SRAM_OE_N), // .OE_N
23   .sram_external_interface_WE_N (SRAM_WE_N), // .WE_N
24   .sdram_wire_addr  (DRAM_ADDR), // sdram_wire.addr
25   .sdram_wire_ba    ({DRAM_BA_1,DRAM_BA_0}), // .ba
26   .sdram_wire_cas_n (DRAM_CAS_N), // .cas_n
27   .sdram_wire_cke   (DRAM_CKE), // .cke
28   .sdram_wire_cs_n  (DRAM_CS_N), // .cs_n
29   .sdram_wire_dq     (DRAM_DQ), // .dq
30   .sdram_wire_dqm   ({DRAM_UDQM,DRAM_LDQM}), // .dqm
31   .sdram_wire_ras_n (DRAM_RAS_N), // .ras_n
32   .sdram_wire_we_n  (DRAM_WE_N), // .we_n
33 );
34 endmodule

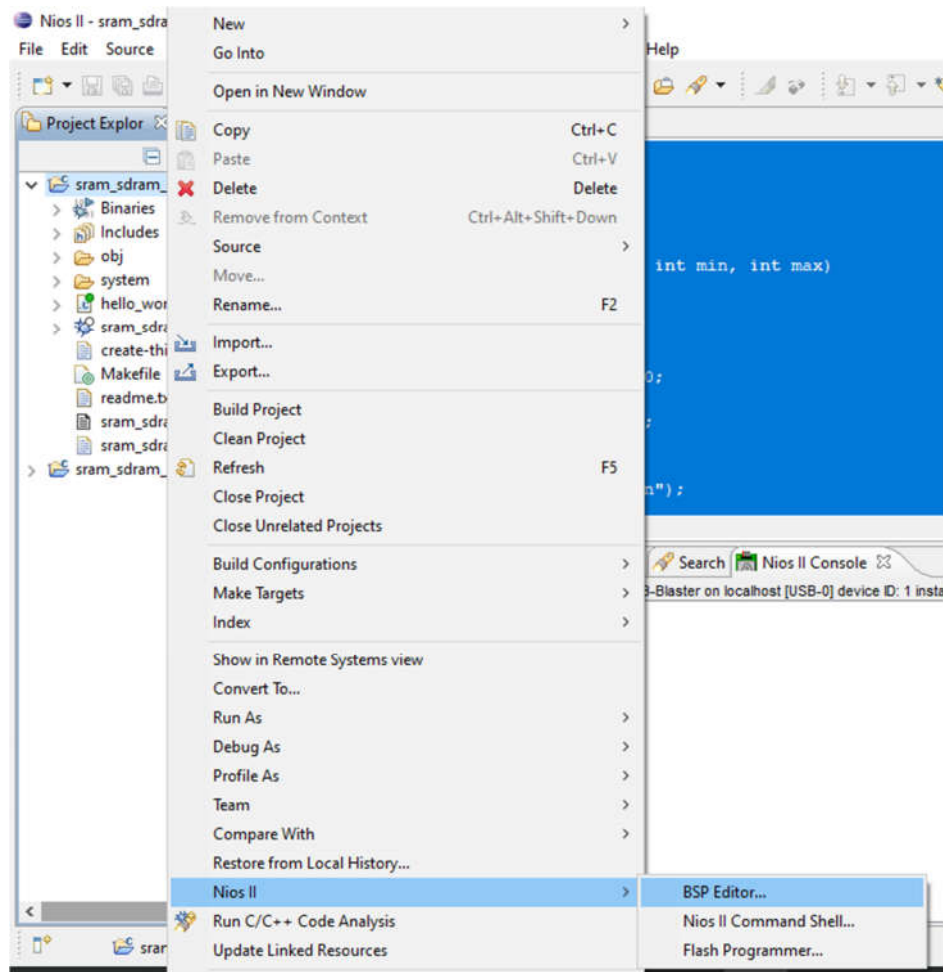
```

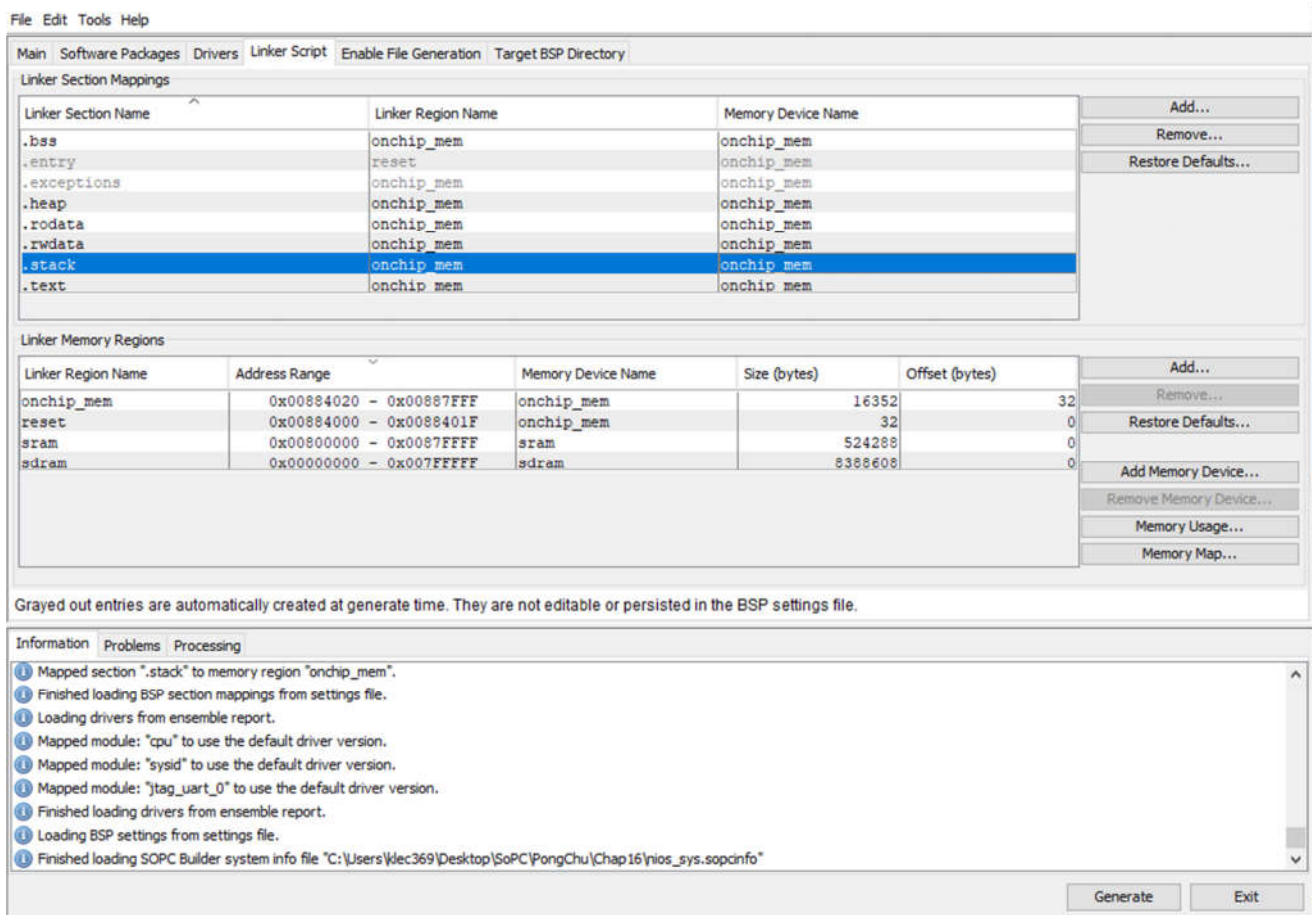
Biên dịch code, và nạp file SOF lên KIT.

❖ Tạo Project trong Eclipse: “Hello World Small”

Như vậy, trong phần cứng chúng ta có 3 Memory: On-chip Memory, SDRAM và SRAM.

Trong Eclipse thực hiện cài đặt BSP Editor cho projects để cấu hình cho đoạn code của chương trình sẽ được lưu đúng trên bộ nhớ **on-chip Memory**.





❖ **Thực hiện và biên dịch đoạn code Check Memory: ghi và đọc data trên SRAM và SDRAM như sau:**

```
#include "sys/alt_stdio.h"
#include <stdio.h>
#include <alt_types.h>
#include "system.h"

int check_mem(alt_u32 mem_base, int min, int max)
{
    int err, real_err;
    alt_u32 *pbase;
    alt_u32 i;
    alt_u32 t_pattn = 0xfa30fa30;

    pbase = (alt_u32 *)mem_base;
    err = 0;

    printf("write started ... \n");
    for(i=min; i<(max-3);i++){
        pbase[i] = i ^ t_pattn;
        //pbase[i] = i + 9;
        //printf("Data %d is %d \n",i,pbase[i]);
    }

    for (i=max-3; i<=max;i++){
        pbase[i] = i;
    }

    printf("read back started ... \n");
    for (i=min; i<=max; i++){
```

```

        //printf("Data %d is %d \n",i,pbase[i]);
        if(pbase[i] != (i^t_pattn)){
            err++;
        }
    }

    real_err = err-4;
    printf("completed with %d actual errors.\n", real_err);
    return(real_err);
}

int main()
{
    alt_putstr("Hello from Nios II!\n");

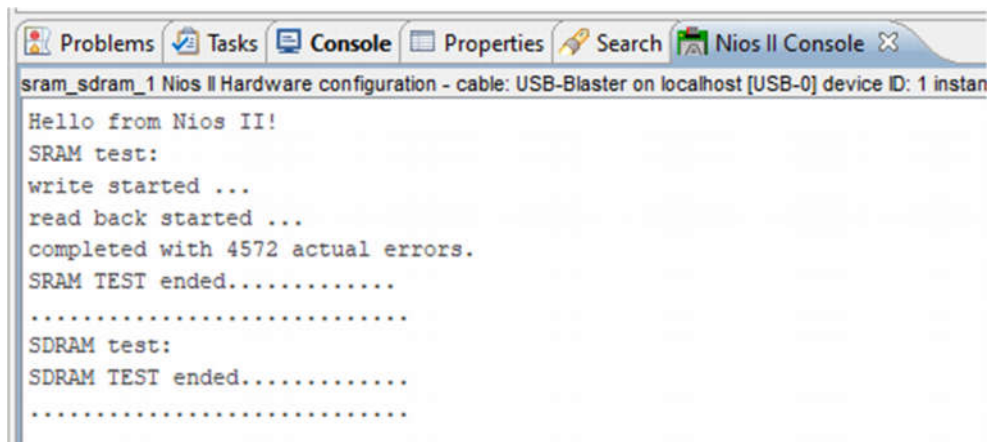
    printf("SRAM test: \n");
    check_mem(SRAM_BASE,0,0x0001ffff);
    //check_mem(SRAM_BASE,0,0x00020000);
    printf("SRAM TEST ended.....\n");
    printf(".....\n");

    printf("SDRAM test: \n");
    check_mem(SDRAM_BASE,0,0x001fffff);
    printf("SDRAM TEST ended.....\n");
    printf(".....\n");
    /* Event loop never exits. */
    while (1);

    return 0;
}

```

Kết quả thực hiện thành công



BÀI TẬP

Bài 1. Thực hiện bài tập 8 Switch bật 8 Led. Yêu cầu phần cứng như sau (không sử dụng bộ nhớ On-chip Memory):

- **Nios II/e**
- **Clock**
- **Jtag-UART**
- **SDRAM**
- **ALTPLL**
- **GPIO – Switch**
- **GPIO - Led**